STAFF REVIEW · H446 034 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q4

Do Now — source answers

1. Bucket 0: (6 + 1) MOD 7 = 0, so probing wraps around.

2. False. The age test is true but hasPermission is false, so the AND expression is false.

3. It becomes the left child of the root’s right child.

4. Use a queue: FIFO serves the earliest job first. A stack uses LIFO and would process the most recently added job first.

I Do — source worked steps

1. Check blank input — CODE:
```
Console.Write("New username: ");
string username = (Console.ReadLine() ?? "").Trim();
if (username.Length == 0)
{
Console.WriteLine("Username required");
}
```

2. Check whether the username exists — Use the lookup function from Lesson 033. Do not register duplicates, including usernames moved by probing.

3. Enforce a size limit — Track how many table buckets are occupied. If all seven are full, refuse a new account gracefully.

We Do — guided CHECK

1. Protect against duplicates
CODE:
```
if (username.Length == 0)
Console.WriteLine("Username required");
else if (FindUser(username) >= 0)
Console.WriteLine("Username exists");
else
Console.WriteLine("Ready to register");
```

2. Explain your solution first
How do we prove the collision handler works?
Test a collision
Register 'ab' and 'ba'. Check both receive distinct buckets and FindUser locates both.

3. Explain your solution first
Which structure supports FIFO processing? Provide three plausible alternatives and justify the correct response.
Build a synoptic MCQ

Queue is correct. A stack is LIFO; a binary search tree organises ordered keys; a hash table is optimised for key-based lookup.

You Do — source mark schemes

1. CODING 1 · COMPLETE REGISTER
Register: reject empty usernames; look up existing names first; reject duplicates; check for free buckets; insert using the same hash and linear-probing routine from Lesson 033. A full table must exit cleanly.

2. CODING 2 · COMPLETE LOGIN
First FindUser(name). If no match, access denied. If found, compare the supplied demonstration password against the associated DEMO value for that user. This is a classroom simulation: plaintext password comparisons and the simple table hash are not secure production practices.

3. CODING 3 · TEST TABLE
Sample tests: registered username+correct demo password → accept; correct username+wrong demo password → deny; unknown name → deny; duplicate name → refuse; 'ab'+'ba' → store and retrieve both; all 7 buckets occupied → reject extra registration without hanging. Add actual outcomes.

4. SYNOPTIC MCQ ASSESSMENT
MCQ marking: cover exactly the structures in the lesson specification. Array indexing/dimensions; records vs tuples; list resizing; linked-list pointers; directed graph edges; LIFO/FIFO; tree hierarchy and BST ordering; hash keys and collisions. Check each distractor and correct misconceptions.

Exit Ticket — source answer

1. False. Two keys can hash to the same index. Collision resolution, such as linear probing, prevents one from simply overwriting the other.